

Relatório

1)

A comunicação foi implementada via HTTP/REST com HTTPS. O API Gateway é o único ponto de entrada, o cliente nunca acessa os serviços diretamente. O gateway recebe a requisição e a encaminha para o serviço correto usando o `https.request`. O serviço de Pedidos também se comunica diretamente com o serviço de Produtos para validar se o produto existe antes de criar o pedido, Todos os dados são trocados em formato JSON.

2)

A estratégia adotada foi consistência eventual com propagação síncrona sequencial. Quando ambas as réplicas estão disponíveis o gateway escreve primeiro no serviço primário (5002) e aguarda a confirmação da réplica (5012) antes de retornar ao cliente. Se a réplica responde com erro ela é imediatamente marcada como indisponível e o sistema degrada para consistência eventual, ou seja a escrita persiste apenas no primário e a réplica ficará com dados desatualizados até se recuperar. Quando volta ao ar o gateway detecta a recuperação via heartbeat e aciona automaticamente o mecanismo de sincronização que envia todos os dados pendentes. Essa escolha foi intencional pois alinhado ao Teorema CAP que afirma que um sistema distribuído nunca pode garantir consistência, disponibilidade e tolerância a partição ao mesmo tempo, optei por priorizar disponibilidade pois indisponibilidade total seria mais prejudicial ao usuário do que dados temporariamente desatualizados em uma réplica.

3)

O sistema continua funcionando parcialmente quando o Serviço de Pedidos cai. O API Gateway detecta a falha via heartbeat em até 10 segundos sendo 2 tentativas com intervalo de 5 segundos cada, registra a falha em log com timestamp e passa a retornar 503 para requisições de pedidos. Os Serviços de Usuários e Produtos continuam operando normalmente sem nenhum impacto, login, registro, listagem e criação de produtos seguem funcionando. Quando o Serviço de Pedidos volta o gateway registra a recuperação em log e retoma o roteamento normalmente de forma automática

4)

No login, o servidor gera um token JWT assinado com uma chave secreta, contendo o campo role (user ou admin). Ao criar um produto, o serviço valida o token e verifica o role, se não for admin, retorna 403. Como o token é assinado, não pode ser alterado pelo cliente. Todo usuário cadastrado recebe role: user automaticamente, sem possibilidade de se auto-promover.

5)

Banco de dados em arquivos JSON: não suporta acessos concorrentes com segurança. Em produção, poderia ser utilizado o PostgreSQL, MongoDB ou similar.

Certificado SSL autoassinado: navegadores e clientes não confiam automaticamente. Em produção, usaria certificados emitidos por autoridades reconhecidas como Let's Encrypt.

Gateway como ponto único de falha: se o gateway cair, todo o sistema para. Em produção, haveria múltiplas instâncias com load balancer.

Sem rollback na replicação: se a escrita no primário funcionar mas a réplica falhar, o dado fica temporariamente inconsistente até o sync. Em produção, poderia utilizar transações distribuídas ou padrão Saga.

Sem autenticação mútua entre serviços (mTLS): a chave interna é simples. Em produção, usaria mTLS ou service mesh como Istio.